SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Naveen Kumar S	**Reg. No.:**	192421030
Section / Batch:	B Tech IT	**Date:**	28th July 2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q25. Debugging Closest Pair Code (Incomplete Distance Computation)

The following brute force Closest Pair implementation computes distance incorrectly because one coordinate component is ignored. Carefully analyze the formula and identify why the returned minimum distance is inaccurate. Debug the distance calculation step-by-step to include all required terms correctly. Optimize the implementation by minimizing repeated access to point coordinates if possible. Analyze the time complexity of the corrected solution. Rewrite the final code with proper structure and comments.

```
import math
def closest_pair(points):
  min_dist = float('inf')
  for i in range(len(points)):
    for j in range(i+1, len(points)):
      dist = math.sqrt((points[i][0]-points[j][0])**2)
```

```
        if dist < min_dist:
            min_dist = dist
    return min_dist
```

Code of Conduct

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Naveen Kumar S

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Introduction

The **Closest Pair of Points** problem is a classical computational geometry problem in which the objective is to determine the minimum Euclidean distance between any two points in a two-dimensional plane. It has applications in computer graphics, robotics, navigation systems, geographic information systems (GIS), machine learning, clustering, and collision detection.

The given Python program attempts to solve this problem using the **Brute Force** approach. In the brute-force method, every point is compared with every other point, and the smallest distance is returned.

However, the supplied implementation contains a major logical error in the distance calculation. The algorithm considers only the **x-coordinate difference** while completely ignoring the **y-coordinate difference**. As a result, the returned distance is often incorrect and does not represent the true Euclidean distance between two points.

This report analyses the error, explains the debugging process, presents the corrected implementation, discusses possible optimizations, analyses the time complexity, verifies correctness using an example, and concludes with observations on the efficiency of the brute-force solution.

---

2. Problem Statement

Given a collection of two-dimensional points, determine the smallest Euclidean distance between every possible pair of points.

The provided code is:

```
import math

def closest_pair(points):
    min_dist = float('inf')

    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2)

            if dist < min_dist:
                min_dist = dist

    return min_dist
```

Although this program appears correct at first glance, it computes the distance using only the x-coordinate.

---

3. Understanding the Closest Pair Problem

Suppose we have the following points:

(2,3)
(5,7)
(4,4)
(8,2)

The goal is to compare every possible pair.

The algorithm checks

- Point1 ↔ Point2
- Point1 ↔ Point3
- Point1 ↔ Point4
- Point2 ↔ Point3

- Point2 ↔ Point4
- Point3 ↔ Point4

The smallest distance among all comparisons is returned.

---

4. Understanding Euclidean Distance

The Euclidean distance between two points

(x1,y1)(x_1,y_1)(x1,y1)

and

(x2,y2)(x_2,y_2)(x2,y2)

is

Distance=(x2−x1)2+(y2−y1)2Distance=\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}Distance=(x2−x1)2+(y2−y1)2

Both coordinate differences are equally important.

The x-distance alone does not represent the actual distance unless both points lie on the same horizontal line.

---

5. Identifying the Bug

The incorrect statement is

dist = math.sqrt((points[i][0]-points[j][0])**2)

Observe carefully.

The code calculates only

(x1−x2)²

The following component is missing

(y1−y2)²

Therefore,

$\sqrt{((x1-x2)^2)}$

is calculated instead of

$\sqrt{((x1-x2)^2+(y1-y2)^2)}$

The result is simply the absolute horizontal distance.

---

6. Why is the Result Incorrect?

Consider two points

A = (2,3)

B = (2,9)

Actual distance

$\sqrt{((2-2)^2+(3-9)^2)}$

$= \sqrt{36}$

$= 6$

The given program computes

$\sqrt{((2-2)^2)}$

$=0$

which is completely wrong.

The program incorrectly assumes both points occupy the same position.

---

7. Another Example

Consider

A=(1,1)

B=(4,5)

Correct distance

$\sqrt{((4-1)^2+(5-1)^2)}$

$=\sqrt{25}$

$=5$

Incorrect program

$\sqrt{((4-1)^2)}$

$=3$

Difference

Correct = 5

Wrong = 3

The error becomes significant as the vertical difference increases.

---

8. Step-by-Step Debugging Process

Step 1

Locate the distance formula.

dist = math.sqrt((points[i][0]-points[j][0])**2)

Step 2

Notice only index **0** is accessed.

points[i][0]

points[j][0]

Index 0 refers only to x-coordinate.

Step 3

Add the missing y-coordinate.

points[i][1]
points[j][1]

Step 4

Compute x difference

dx = points[i][0]-points[j][0]

Step 5

Compute y difference

dy = points[i][1]-points[j][1]

Step 6

Apply Euclidean formula

dist = math.sqrt(dx*dx + dy*dy)

Now the distance is mathematically correct.

---

9. Debugging Summary

Issue	Explanation	Solution
Missing y-coordinate	Only x-distance calculated	Include y difference
Wrong formula	Horizontal distance only	Euclidean distance

Incorrect minimum distance	Comparisons inaccurate	Correct formula
Incorrect output	Does not represent actual geometry	Use both coordinates

---

10. Optimizing the Implementation

The original implementation repeatedly accesses

points[i][0]

points[i][1]

points[j][0]

points[j][1]

These repeated indexing operations can be minimized.

Instead,

x1,y1=points[i]

x2,y2=points[j]

Now each coordinate is accessed only once.

Advantages

- Cleaner code
- Better readability
- Slightly fewer indexing operations
- Easier debugging

---

11. Corrected Algorithm

1. Initialize minimum distance as infinity.
2. Compare every pair of points.
3. Extract coordinates.

4. Compute x difference.
5. Compute y difference.
6. Calculate Euclidean distance.
7. Update minimum distance.
8. Return minimum distance.

---

12. Final Corrected Code

```
import math

def closest_pair(points):
    """
    Returns the minimum Euclidean distance
    between any two points using the
    brute-force approach.
    """

    # Initialize minimum distance
    min_dist = float('inf')

    # Compare every pair of points
    for i in range(len(points)):
        for j in range(i + 1, len(points)):

            # Extract coordinates once
            x1, y1 = points[i]
            x2, y2 = points[j]

            # Compute coordinate differences
            dx = x1 - x2
            dy = y1 - y2

            # Compute Euclidean distance
            dist = math.sqrt(dx * dx + dy * dy)

            # Update minimum distance
            if dist < min_dist:
                min_dist = dist

    return min_dist
```

```
# Example usage
points = [(2, 3), (5, 7), (4, 4), (8, 2)]

print("Minimum Distance =", closest_pair(points))
```

---

13. Further Optimization

Python provides

math.hypot(dx, dy)

Instead of

math.sqrt(dx*dx+dy*dy)

This improves readability.

Example

dist = math.hypot(dx, dy)

---

14. Example Execution

Input

points=[
(2,3),
(5,7),
(4,4),
(8,2)
]

Comparisons

Pair	Distance
(2,3)-(5,7)	5.000
(2,3)-(4,4)	2.236

(2,3)-(8,2) 6.083

(5,7)-(4,4) 3.162

(5,7)-(8,2) 5.831

(4,4)-(8,2) 4.472

Minimum distance

2.236

---

15. Correctness Verification

The corrected algorithm satisfies the Euclidean distance formula by considering both the horizontal (**x**) and vertical (**y**) differences. Every unique pair of points is evaluated exactly once, ensuring that no possible minimum-distance pair is missed. After computing the distance for each pair, the algorithm updates the minimum distance whenever a smaller value is found. Therefore, the final result returned by the algorithm is guaranteed to be the minimum Euclidean distance among all pairs of points.

---

16. Time Complexity Analysis

Assume there are **n** points.

Outer Loop

Runs **n** times.

Inner Loop

Runs approximately

n−1

n−2

...

1

Total comparisons

n(n−1)2\frac{n(n-1)}{2}2n(n−1)

Therefore

Time Complexity

Case	Complexity
Best Case	O(n²)
Average Case	O(n²)
Worst Case	O(n²)

Unlike search algorithms, every pair must be checked regardless of the data. Hence, the complexity remains **O(n²)** in all cases.

---

17. Space Complexity

The algorithm stores only a few additional variables:

- `min_dist`
- `dx`
- `dy`
- `dist`
- coordinate variables (`x1`, `y1`, `x2`, `y2`)

No extra data structures proportional to the input size are created.

Space Complexity = O(1) (constant extra space).

---

18. Advantages of the Corrected Brute-Force Approach

- Simple and easy to understand.

- Straightforward implementation.
- Works correctly for any set of 2D points.
- No preprocessing or sorting required.
- Suitable for small datasets.
- Uses constant extra memory (O(1)).
- Useful as a baseline solution for testing more advanced algorithms.

---

19. Limitations

- Performs a large number of comparisons as the number of points increases.
- Time complexity remains **O(n²)** even in the best case.
- Not suitable for very large datasets containing thousands or millions of points.
- Slower than divide-and-conquer closest pair algorithms, which can achieve **O(n log n)** time complexity.
- Cannot terminate early because every pair must be examined.

---

20. Applications of the Closest Pair Problem

The closest pair algorithm has numerous practical applications, including:

- Collision detection in computer graphics and gaming.
- Geographic Information Systems (GIS) for finding nearby locations.
- Robotics for obstacle avoidance and navigation.
- Wireless sensor networks for identifying neighbouring sensors.
- Machine learning and clustering algorithms.
- Astronomy for analysing the proximity of stars and celestial objects.
- Computer vision and image processing.
- Logistics and route planning.

---

21. Conclusion

The original brute-force implementation contained a significant logical error by considering only the **x-coordinate** when computing the distance between two points. This resulted in inaccurate minimum-distance calculations because the **y-coordinate** contribution was completely ignored. By debugging the algorithm step by step, introducing the missing **y-coordinate difference**, and applying the

complete Euclidean distance formula, the implementation now correctly computes the shortest distance between every pair of points.

The code was further improved by reducing repeated coordinate indexing, making it cleaner and slightly more efficient while preserving the brute-force approach. Although the corrected solution still requires **$O(n^2)$** time because every pair of points must be examined, it is accurate, easy to understand, and appropriate for small datasets. For larger datasets, more advanced approaches such as the divide-and-conquer closest pair algorithm can significantly improve performance to **$O(n \log n)$**.

This content is sufficiently detailed to span approximately 6–7 pages in Microsoft Word using Times New Roman, 12 pt font, 1.5 line spacing, with standard margins.